MakeMyMarrige

Wedding Workspace
UI/UX Developer Handoff

Version 1.0 · 5 November 2026 · Implementation baseline

Modern Indian Minimalism · Next.js + TypeScript · NestJS · MongoDB Atlas

1. Purpose and maturity

This package translates the agreed Wedding Workspace UX direction and business rules into a developer starting specification. It defines route inventory, component responsibilities, design tokens, responsive behaviour, interaction states, permissions, acceptance criteria and implementation order.

Maturity note: Prior visual boards are concept references, not individual Figma frames. This is an implementation specification; it does not claim every screen has been individually drawn, usability-tested or implemented. Reconcile exact API operations and schemas against the authoritative OpenAPI file before wiring services.

2. Product and architecture baseline

• Frontend: Next.js App Router + TypeScript, installable PWA.

• Backend: NestJS + TypeScript REST API, modular monolith.

• Data: MongoDB Atlas; shared database/shared collections for pilot with logical tenant isolation.

• Queue/cache: managed Redis + BullMQ; Redis is not source of truth.

• Media: private Cloudflare R2; presigned uploads and short-lived authorised downloads.

• API: REST, HTTPS, JSON, OpenAPI 3.1, /v1.

• Tenant resolved from authenticated membership; never trust client weddingId.

• UTC instants; wedding-local semantics Asia/Kolkata.

• Money stored as integer paise, INR.

3. Design system

Token	Value	Usage
Background	#FAF8F4	Warm ivory
Surface	#FFFFFF	Cards
Primary	#7A2E42	Actions/navigation
Accent	#BD9560	Restrained highlights
Text	#282522	Main content
Muted	#77716B	Supporting
Border	#EAE4DA	Separators
Semantic	#287A52 / #A96816 / #B33A36	Success/warning/danger

Typography: Inter for operational UI; Playfair Display for selective editorial headings; Noto Sans Devanagari for Devanagari. 4px spacing base, 12px card radius, 8px control radius. Breakpoints: mobile <640px; tablet 640–1023px; desktop >=1024px. Desktop persistent sidebar; tablet collapsible drawer; mobile compact top bar and bottom navigation. Cultural motifs remain restrained.

4. Module and screen inventory

01. AUTH & ONBOARDING

Routes: /login, /register, /verify-email, /onboarding

Purpose: Account access, email verification, workspace creation and co-owner invitation.

Components: AuthLayout, AuthForm, PasswordField, VerificationStatus, OnboardingStepper

Rules: Account is verified before workspace creation or invitation acceptance. Secure HttpOnly cookies; refresh rotation; generic auth errors.

Priority: P0

02. DASHBOARD

Routes: /app

Purpose: Needs-attention-first overview, planning metrics, upcoming functions and recent activity.

Components: WorkspaceShell, PageHeader, NeedsAttentionPanel, MetricCard, UpcomingFunctions, ActivityFeed, QuickActions

Rules: Finance metrics are permission-gated. Aggregates are tenant-scoped. Every quick action checks permission.

Priority: P0

03. FUNCTIONS

Routes: /app/functions, /app/functions/[functionId]

Purpose: List, create, edit, review, publish and cancel wedding functions.

Components: FunctionList, FunctionCard, FunctionForm, ScheduleFields, VenuePicker, PublicationStatus, SnapshotViewer

Rules: Draft → pending approval → published → cancelled. Publish/update/cancel require dual co-owner approval. Guests see published snapshots only.

Priority: P0

04. TASKS

Routes: /app/tasks, /app/tasks/[taskId]

Purpose: Task dashboard, board, task details, assignment, checklist and dependencies.

Components: TaskBoard, TaskColumn, TaskCard, TaskForm, FunctionSelect, AssigneePicker, ChecklistEditor, DependencyPicker, ActivityTimeline

Rules: Every task links to a function or explicitly to General wedding planning. One responsible owner; multiple assignees. Mandatory checklist and dependencies gate completion. Reopen requires reason.

Priority: P0

05. GUESTS & HOUSEHOLDS

Routes: /app/guests, /app/guests/[guestId], /app/households

Purpose: Individual guest directory, household grouping, corrections and merge review.

Components: GuestTable, GuestForm, HouseholdPanel, GuestProfile, CorrectionReview, MergeComparison

Rules: Guest is an individual identity. Household is administrative grouping only. Requester cannot approve own correction. Merge requires dual co-owner approval.

Priority: P0

06. RSVP

Routes: /app/rsvp, /app/rsvp/functions/[functionId]

Purpose: Function-wise response dashboard, individual responses, reminders and additional guest requests.

Components: RsvpSummary, FunctionResponseTable, GuestResponseCard, DeadlineBanner, ReminderComposer, ExtraGuestRequestReview

Rules: One RSVP per guest/function. Partial household responses. Deadline enforced server-side. Extra guest request does not create a guest automatically.

Priority: P0

07. BUDGET & EXPENSES

Routes: /app/finance, /app/finance/expenses/[expenseId]

Purpose: Budget allocation, expense submission, approvals, payments and reports.

Components: BudgetSummary, CategoryBreakdown, ExpenseTable, ExpenseForm, ApprovalProgress, PaymentLedger, FinanceChart

Rules: Money stored as integer paise. Treasurer approves normal expenses. Treasurer-created/beneficiary expenses require both co-owners. Payment entries and corrections are append-only.

Priority: P1

08. VENDORS

Routes: /app/vendors, /app/vendors/[vendorId]

Purpose: Discovery, profiles, shortlist, comparison, quotes, selection and booking.

Components: VendorSearch, VendorCard, VendorProfile, ComparisonTable, QuotePanel, BookingForm, VendorStatusTimeline

Rules: Discovered/shortlisted/compared/selected/booked/completed remain distinct. Only co-owners select/remove selected vendors. Selection is not booking; quote is not contract.

Priority: P1

09. DIGITAL SHAGUN

Routes: /app/shagun

Purpose: UPI destination governance, guest contributions, provider verification and reconciliation.

Components: UpiDestinationList, DualApprovalPanel, ContributionTable, PaymentStateBadge, ReconciliationSummary

Rules: No platform-held funds. Only dual-approved UPI destinations. Server-side provider verification; signed webhooks, idempotency and replay protection. Client redirects/screenshots are not proof.

Priority: P1

10. GALLERY & PHOTOS

Routes: /app/gallery, /app/gallery/albums/[albumId]

Purpose: Albums, upload, moderation, photo details and approved photo viewing.

Components: AlbumGrid, PhotoGrid, UploadDropzone, UploadQueue, ModerationQueue, PhotoLightbox, PhotoMetadata

Rules: Private R2; presigned uploads; guest upload only to invited functions; guest uploads moderated; eligible member uploads auto-approved after technical validation; strip GPS EXIF.

Priority: P1

11. APPROVALS & GOVERNANCE

Routes: /app/approvals, /app/governance

Purpose: Proposal inbox, immutable snapshots, dual approval and governance history.

Components: ApprovalQueue, ProposalDetail, SnapshotDiff, ApprovalProgress, DecisionComposer, GovernanceRoleCard

Rules: Two distinct co-owner accounts for dual approval. Requester cannot approve own proposal. Approval applies to exact immutable snapshot/version. Expiry/rejection/change requests are explicit.

Priority: P0

12. TEAM & PERMISSIONS

Routes: /app/team, /app/team/[memberId]

Purpose: Organiser directory, invitations, permission scopes, Treasurer and membership status.

Components: MemberTable, InviteMemberForm, RoleBadge, PermissionMatrix, FunctionScopePicker, MemberDetail, RevokeAccessDialog

Rules: Organiser access explicitly granted and scope-bound. Only co-owners invite/change/remove organisers. One active Treasurer. Suspension immediately revokes access; suspended membership occupies membership slot.

Priority: P1

13. NOTIFICATIONS

Routes: /app/notifications, /app/activity

Purpose: Actionable inbox, activity feed, preferences and reminders.

Components: NotificationInbox, NotificationItem, NotificationDetail, CategoryFilter, PreferenceMatrix, ReminderSettings

Rules: Role-aware content. Deep links recheck permissions. Mark-as-read does not mutate business state. Avoid duplicate alerts; critical reminders not silently suppressed.

Priority: P1

14. AUDIT & SECURITY

Routes: /app/audit, /app/security, /app/security/sessions

Purpose: Immutable audit history, security events, session management and export.

Components: AuditTable, AuditFilters, AuditEventDetail, SecurityAlertList, SessionTable, RevokeSessionDialog, ExportRequest

Rules: Audit captures actor, role, action, target, timestamp, outcome and correlation ID. Never log secrets. Audit export requires dual co-owner approval. Session revocation is server-enforced.

Priority: P1

15. WORKSPACE SETTINGS

Routes: /app/settings

Purpose: Workspace profile, preferences, privacy, integrations and lifecycle controls.

Components: SettingsNav, WorkspaceProfileForm, PersonalProfileForm, NotificationPreferences, AppearanceSettings, IntegrationStatus, LifecyclePanel

Rules: Integrations secrets remain server-side. Cancellation, deactivation and deletion are distinct. Deletion timing/backup retention must follow approved lifecycle policy and verified infrastructure.

Priority: P1

16. GUEST PORTAL ENTRY (WORKSPACE LINKAGE)

Routes: /guest/* (separate surface)

Purpose: Workspace-controlled published information and guest-facing entry points.

Components: GuestLinkPreview, PublicationPreview, GuestAccessStatus

Rules: Guest portal is a separate experience and not part of this workspace handoff; include only preview links and permission-aware navigation from workspace.

Priority: Out of workspace scope

5. Shared component library

Foundations

Button, IconButton, Input, Textarea, Select, Combobox, DatePicker, Checkbox, RadioGroup, Switch, Tabs, Badge, Avatar, Tooltip, Popover, Dialog, Drawer, DropdownMenu, Separator, Skeleton, Progress, Toast, Alert, EmptyState, ErrorState, Pagination, Breadcrumbs.

Data display

DataTable, MobileRecordList, MetricCard, StatusBadge, Timeline, ActivityFeed, FilterBar, SearchInput, DateRangeFilter, MoneyDisplay, CurrencyInput, UserCell, FunctionTag, ApprovalIndicator, PermissionGate.

Workspace-specific

WorkspaceShell, WorkspaceSidebar, MobileWorkspaceNav, PageHeader, NeedsAttentionPanel, FunctionSelector, GuestPicker, HouseholdBadge, AssigneePicker, ChecklistEditor, DependencyPicker, ApprovalSnapshot, DualApprovalProgress, ExpenseStatus, VendorLifecycleBadge, RsvpStatus, PhotoModerationCard, AuditEventRow.

6. Interaction and state standards

• Client validation for UX; server validation authoritative; preserve input on recoverable errors.

• Prevent duplicate submissions; idempotency for eligible creates and financial operations.

• Use version/If-Match for concurrency; never silently overwrite.

• Never optimistically claim payment verification, dual approval, deletion or publication.

• Use inline field errors, toasts and persistent banners appropriately.

• Dialogs require focus management and keyboard handling.

• Deep links re-check authorisation.

• Dates display in Asia/Kolkata; money persists in integer paise.

• Uploads show validation, progress and retry; server validates files.

7. Permission model

• Co-owner: full governance; dual approval requires two distinct co-owner accounts.

• Treasurer: finance visibility and normal expense approval; cannot approve own/beneficiary expense.

• Organiser: explicitly granted, function/resource-scoped permissions; no implicit authority.

• Guest: separate guest identity/session; no workspace directory management.

• Support/admin: break-glass only, audited.

• API enforcement is authoritative; suspended membership immediately loses access but occupies membership slot.

8. API integration contract

OpenAPI 3.1 is the source of truth for exact paths, request/response schemas and error envelopes. This handoff deliberately does not invent endpoint names where exact operations have not been reconciled.

• Generate typed API clients from checked-in OpenAPI.

• Shared client handles credentials, CSRF, request IDs, error normalization and refresh policy.

• No tokens in localStorage/sessionStorage; use secure HttpOnly cookies.

• Resolve tenant context from server-side membership/session.

• Cursor pagination default 20, maximum 50; cursor bound to tenant/filter/sort.

• Error envelope: code, message, requestId, details, retryable. Success: data, meta, requestId, servedAt.

• Financial/payment operations idempotent; webhooks signed, replay-protected and deduplicated.

9. Cross-cutting UI states

List pages: Loading/skeleton, populated, empty, filtered-empty, API failure, permission denied, stale cursor, retry

Detail pages: Loading, not found, forbidden, populated, stale version, archived/cancelled, API failure

Forms: Pristine, dirty, validating, submitting, success, field error, server error, conflict, permission denied

Approvals: Pending, first approval, second pending, approved, rejected, changes requested, expired, cancelled, execution failed

Payments: Initiated, pending verification, verified, failed, reversed

Uploads: Selecting, validating, uploading, uploaded-awaiting-moderation, approved, rejected, failed, retrying

10. Acceptance criteria

Dashboard: Attention items precede metrics; finance data permission-gated; aggregates tenant-scoped.

Functions: Drafts private; published snapshot immutable; dual approval for publish/update/cancel; cancellation preserves RSVP/task history.

Tasks: Function association required unless explicitly General wedding planning; one responsible owner; multiple assignees; checklist/dependency gates; reopen requires reason.

Guests: Individual identities distinct from households; requester cannot approve own correction; merge requires dual co-owner approval.

RSVP: One response per guest/function; partial household responses; deadline server-enforced; extra guest request does not auto-create guest.

Finance: Integer paise; Treasurer normal approval; Treasurer conflict-of-interest expense requires both co-owners; payment history append-only.

Vendors: Shortlisted/selected/booked distinct; co-owner-only selection; booking requires agreement/contract record.

Shagun: Dual-approved destination; provider-side server verification; signed/idempotent webhook; no platform-held funds.

Gallery: Private R2; guest upload scoped to invited functions; moderation; GPS EXIF stripping; short-lived authorised downloads.

Approvals: Distinct co-owner accounts; requester cannot approve own proposal; immutable snapshot/version; auditable decisions.

Team: Only co-owners invite/change/remove organisers; one active Treasurer; suspension revokes access immediately.

Notifications: Permission-aware content and deep links; read state does not mutate business data.

Audit: Append-only; actor/time/action/target/outcome/correlation ID; no secrets; export dual-approved.

Settings: Cancellation, deactivation and deletion distinct; lifecycle actions explain irreversibility and timing.

11. Accessibility and quality

Target WCAG 2.2 AA. Require keyboard navigation, visible focus, logical focus order, associated labels/errors, status not conveyed by colour alone, reduced-motion support, 200% zoom and narrow viewport testing. Combine axe-core with manual keyboard and screen-reader review.

12. Testing strategy

• Unit: formatting, validation, state transitions and permission predicates.

• Component: forms, tables, dialogs and all common states.

• Integration: API adapter, auth refresh, CSRF, pagination and errors.

• E2E: onboarding, function publication, task completion, guest correction, RSVP, expense approval, vendor booking, Shagun verification, photo moderation, suspension and lifecycle.

• Security: cross-tenant access, IDOR, role escalation, sessions, CSRF, webhook replay, upload abuse and signed URL expiry.

• Visual regression and manual accessibility testing.

13. Recommended implementation order

1. Design tokens, typography, shell, navigation and primitives.

2. Authentication, session restoration and protected route groups.

3. Workspace onboarding and membership context.

4. Dashboard + Functions + Tasks + Guests + RSVP.

5. Approvals and governance enforcement.

6. Finance + Vendors + Shagun.

7. Gallery + Notifications.

8. Audit + Security + Settings.

9. Responsive/visual QA, accessibility and hardening.

14. Open decisions / release blockers

• OTP provider and delivery contract.

• UPI/payment provider and webhook contract.

• Organiser permission catalogue and scope grammar.

• Photo limits, dimensions, quotas and retention.

• Production/staging URLs and hosting/region.

• Numeric SLOs, rate limits and alert thresholds.

• Legal/regulatory review for payment and data retention.

• Exact API/schema reconciliation against current OpenAPI.

15. Developer start checklist

■ Add design tokens and typography.

■ Build WorkspaceShell and responsive navigation.

■ Build foundational component library.

■ Generate typed API client from authoritative OpenAPI.

■ Implement auth/session/tenant context before feature pages.

- Implement central permission gate.
- Build first vertical slice: Dashboard → Functions → approval workflow.
- Add unit, component, contract and Playwright tests.
- Validate desktop/tablet/mobile and WCAG 2.2 AA.